MINISTÉRIO DA DEFESA • EXÉRCITO BRASILEIRO • IME

Curso de Graduação em Engenharia da Computação

Projeto de Final de Curso • 2026

Planejamento de Trajetória para Robô Terrestre usando Grafos

Apresentação final: da imagem aérea à execução do caminho em ambiente simulado

Autores

Gabriel Dias Vilela • **Tassandro Cavalcante Leitão**

Orientadores

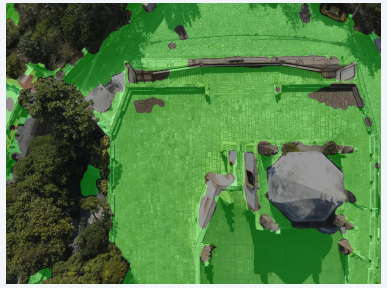
Profa. Claudia Marcela Justel, D.Sc. • Prof. Paulo Fernando Ferreira Rosa, Ph.D.

Rio de Janeiro • 2026

Contexto e motivação

Por que planejar trajetórias de UGV a partir de imagens aéreas

- **Emprego militar e em resgate:** robôs autônomos aumentam a eficácia operacional e reduzem a exposição humana em combate e em eventos extremos (terremotos, chuvas intensas, acidentes radioativos)
- **Planejamento de trajetória:** tarefa essencial à autonomia de robôs terrestres; o ambiente é modelado como um grafo, com vértices nas posições transitáveis e arestas entre posições vizinhas
- **Continuidade:** pesquisa do LIARC sobre cooperação entre UAV e UGV, iniciada em 2023 pelo Maj Fábio Luiz Junior, e PFC 2025 (Almeida e Barbosa, 2025), que planejava caminhos sobre imagens já segmentadas
- **Aplicação potencial:** projeto estratégico Sistemas de Veículos Terrestres Remotamente Pilotados (SVTRP), do CTEx
- **Papel da simulação:** um caminho ótimo sobre uma imagem não garante que o UGV consiga executá-lo; a simulação permite essa verificação sem risco e sem plataforma física



1. O UAV captura a imagem da área e a segmentação identifica as regiões transitáveis



2. O mapa de navegabilidade é convertido em grafo e os algoritmos calculam o caminho



3. O UGV executa o caminho (Clearpath Jackal no Gazebo)

Objetivos

Objetivo principal e objetivos específicos, com o grau de cumprimento

Objetivo principal: construir um sistema modular capaz de, a partir de uma imagem aérea real, identificar as áreas transitáveis, planejar uma trajetória por diferentes algoritmos e executá-la em um UGV, validando-a previamente em ambiente simulado.

✓ **Arquitetura orientada a objetos com cinco módulos encadeados**
Realizado

✓ **RRT sob a mesma interface, comparado a BFS, DFS, Dijkstra e A***
Realizado

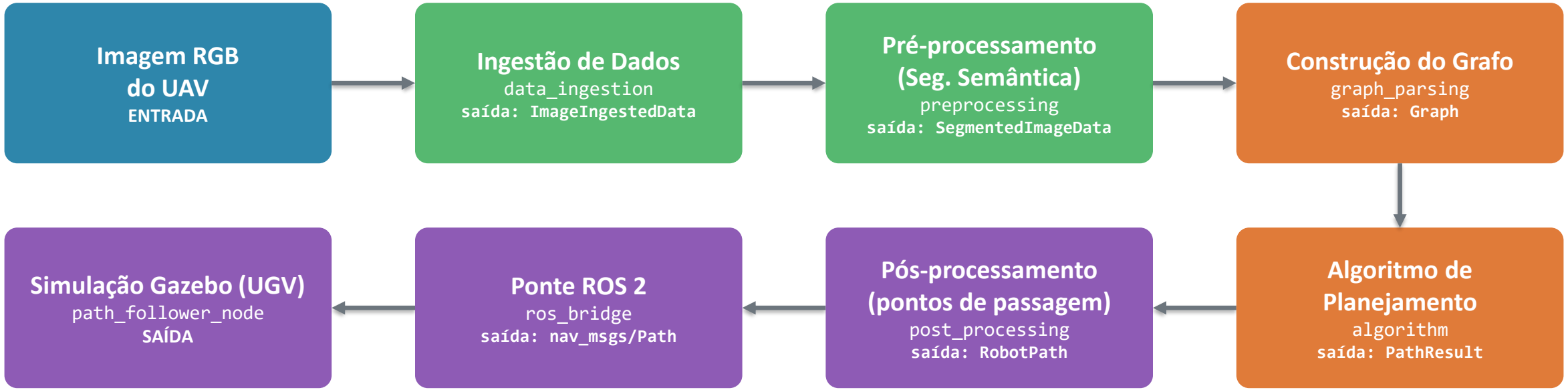
✓ **Segmentação semântica com os modelos pré-treinados (Junior, 2024b)**
Realizado

✓ **Ponte ROS 2 e UGV simulado (Clearpath Jackal) no Gazebo**
Realizado

✓ **Caracterizar os percursos e a correspondência métrica imagem–mundo**
Realizado

Arquitetura do sistema

Cinco módulos independentes em sequência; cada módulo pode ser substituído sem alterar os demais



`IngestedData` → `ImageIngestedData` data_ingestion: dados da imagem recebida do UAV

`SegmentedImageData` preprocessing: mapa de navegabilidade

`Graph` graph_parsing: vértices, arestas e pesos w(e)

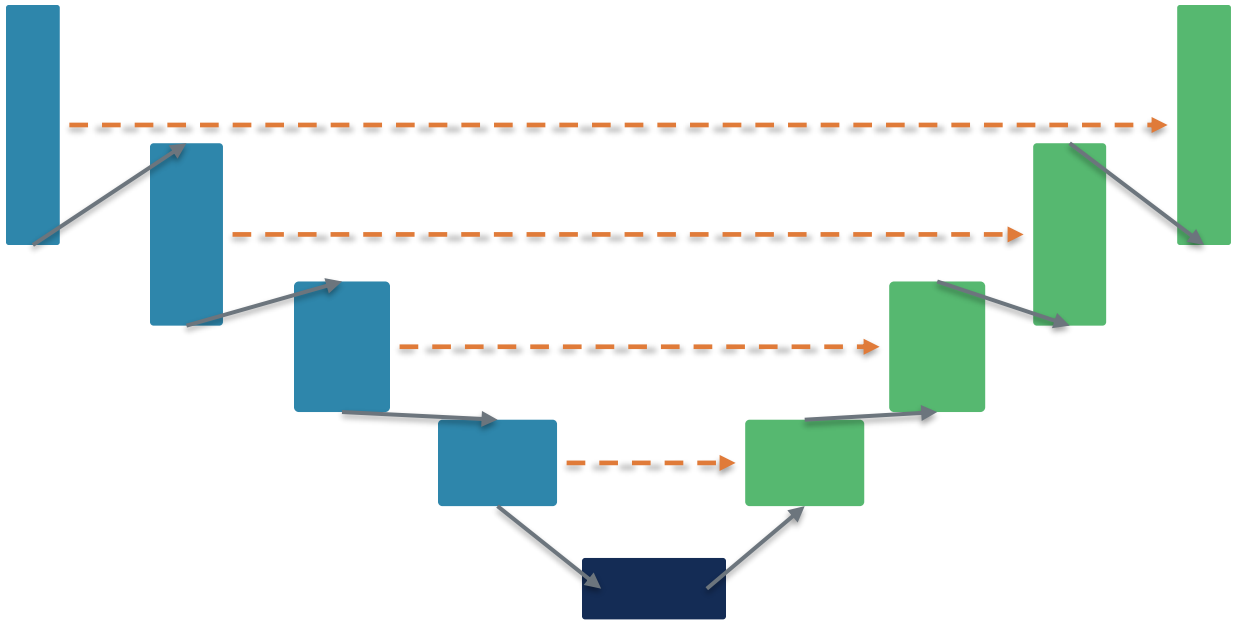
`PathResult` algorithm: vértices do caminho P, custo ℓ(P) e metadados

`RobotPath` post_processing: pontos de passagem e imagens; enviado pela ros_bridge

Cada módulo: interfaces.py, entities.py, index.py (fábrica de objetos) e __init__.py. A substituição de modelo, algoritmo ou formato de saída exige apenas o registro de uma nova classe.

Arquitetura U-Net e mapa de navegabilidade

Ronneberger, Fischer e Brox (2015): rede neural convolucional que classifica cada pixel da imagem



Codificador
reduz a resolução e extrai características

Conexões de atalho
preservam os detalhes espaciais

Decodificador
restaura a resolução

Modelos pré-treinados (Junior, 2024b) no Semantic Drone Dataset (imagens aéreas de 5 a 30 m de altitude, 23 classes), usados apenas em inferência, em substituição à segmentação por cor (HSV) do PFC 2025. Codificadores: ResNet 50/101, EfficientNet-B4/B5 e MobileNet V2.

Mapa de navegabilidade

Formato definido em (Junior, 2025): as regiões transitáveis são representadas por duas cores fixas, BGR (84, 1, 68) e (0, 0, 255); as demais regiões são não transitáveis.

BGR (84, 1, 68)

BGR (0, 0, 255)

Vantagem: o mapa pode ser gerado por qualquer fonte (rede neural, segmentação por cor ou mundo simulado), desde que adote essas cores.

Entrada comum: a construção do grafo e o RRT operam sobre este mapa; a verificação de obstáculo entre dois pontos usa as cores ao longo do segmento.

Comparação dos modelos de segmentação

Imagem aérea do Outeiro da Glória, redimensionada para 1024 × 768 px e processada em GPU integrada

Modelo	mIoU	Acurácia	Área transit. (%)	Tempo (s)
ResNet 50	0,3933	0,8244	65,5	0,72
ResNet 101	0,4032	0,8720	67,5	0,52
EfficientNet-B4	0,5012	0,9120	61,1	0,71
EfficientNet-B5	0,4819	0,9025	69,4	0,57

Tabela 1: mIoU e acurácia divulgadas em (Junior, 2024b); área transitável e tempo medidos neste trabalho. MobileNet V2 omitido por não haver métricas de referência divulgadas.

- **EfficientNet-B4:** maior mIoU (0,5012) e classificação mais conservadora: apenas 61,1% da imagem como transitável, com bordas mais bem delimitadas
- **Tempo inferior a 1 s em todos os modelos:** a diferença de tempo é pequena, e a mIoU é o critério de escolha
- **Área transitável de 61,1% a 69,4%:** o modelo altera a área livre fornecida ao planejador (excesso pode incluir regiões de risco; falta pode bloquear passagens)

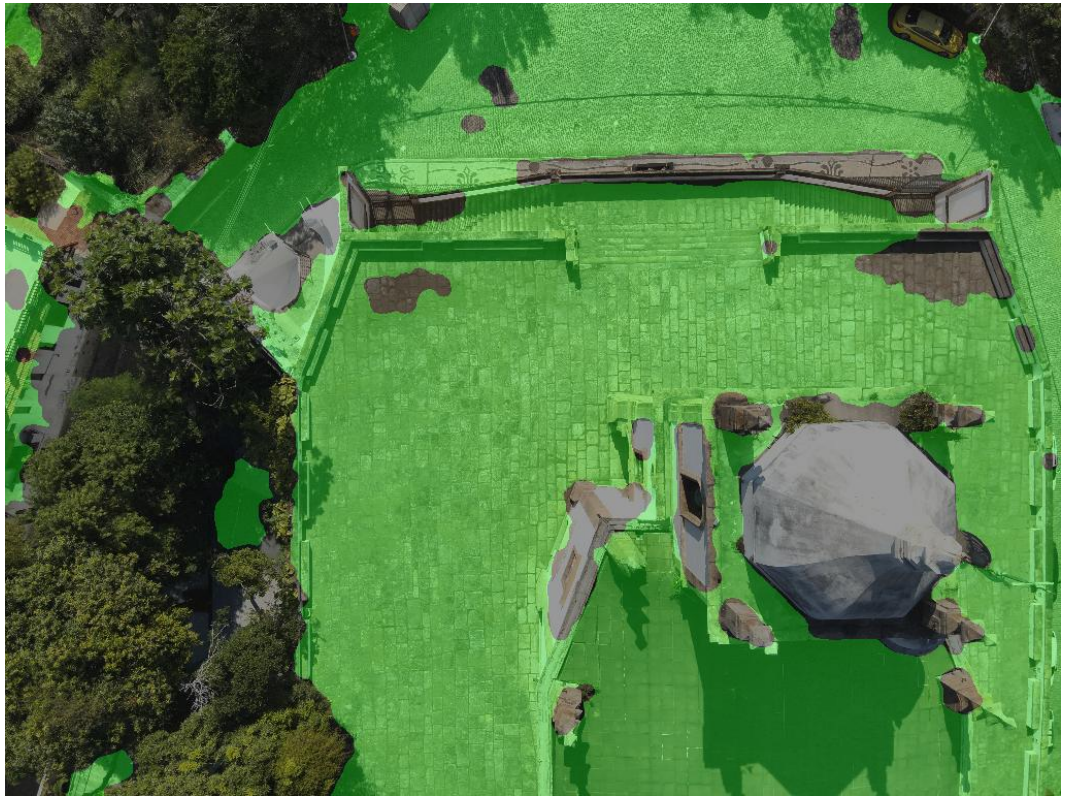
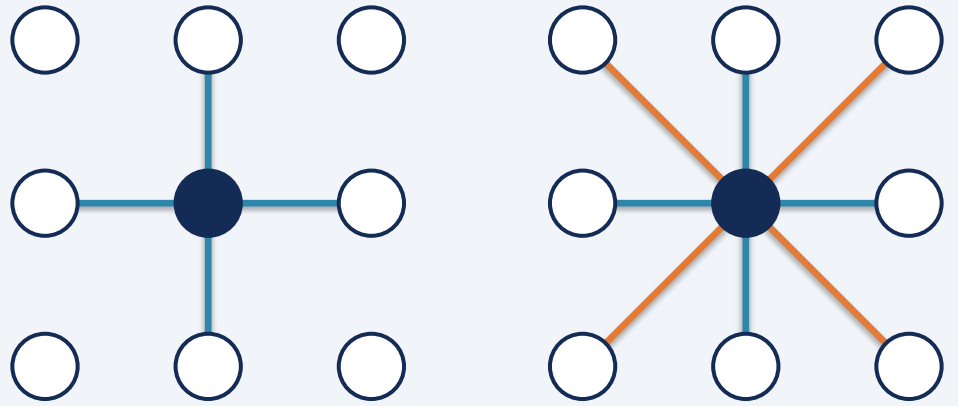


Figura 1: segmentação pela EfficientNet-B4; em verde, a região classificada como transitável. Pátio e vegetação rasteira incluídos; igreja, edificações, árvores densas e veículos excluídos, inclusive em regiões sombreadas.

Construção do grafo

O grafo é obtido do mapa de navegabilidade; a imagem aérea descreve por completo o ambiente de operação do UGV

- **Planejamento global:** todo o ambiente é conhecido antes do início do percurso; no planejamento local, o ambiente é descoberto durante o trajeto
- **Vértices:** pixels classificados como transitáveis no mapa de navegabilidade
- **Arestas:** ligam pixels vizinhos, segundo a conectividade adotada
- **Conectividade 8 (adotada):** permite movimentos diagonais e caminhos mais retos, com cerca de 2× mais arestas; a conectividade 4 gera caminhos em escada
- **Grafo direcionado:** as arestas (u, v) e (v, u) são armazenadas separadamente, pois o peso depende do vértice de chegada
- **Dimensão do problema:** uma imagem de 1000 × 750 px gera centenas de milhares de vértices e milhões de arestas; o tempo de construção do grafo é tão relevante quanto o de busca



círculo: pixel transitável; azul: vizinhos laterais; laranja: vizinhos diagonais

Custo assimétrico: o deslocamento de uma região livre para as proximidades de um obstáculo tem custo maior que o deslocamento inverso; por isso, cada par de vizinhos gera duas arestas, uma em cada sentido.

Função de peso e custo das arestas

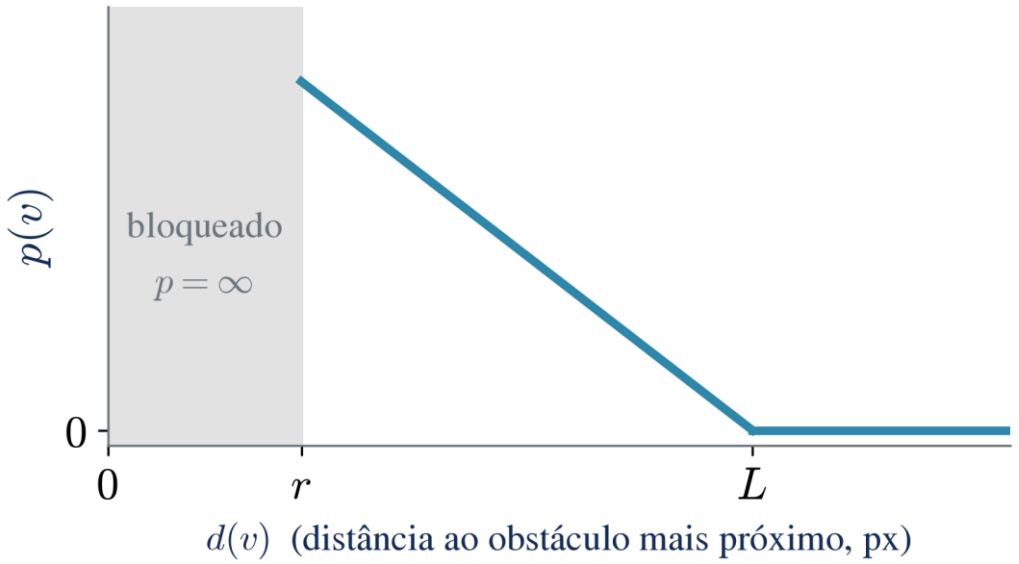
Aplicada no módulo graph_parsing, antes da busca: penaliza trajetórias próximas a obstáculos, sem proibi-las

$$p(v) = \begin{cases} \infty, & \text{se } d(v) \le r \quad \text{(vértice bloqueado)} \\ L - d(v), & \text{se } r < d(v) < L \\ 0, & \text{se } d(v) \ge L \end{cases}$$

d(v): distância do pixel ao obstáculo mais próximo; r (robot_size): folga mínima, igual ao raio do UGV acrescido de uma margem; L (weight_limit): distância a partir da qual o obstáculo deixa de influenciar o peso.

$$w(e) = \delta(e) + p(v), \qquad \delta(e) \in \{1, \sqrt{2}\}$$

- Os pesos w(e) são calculados na construção do grafo e usados por Dijkstra e A*; δ(e) é o comprimento do passo, usado para medir a distância (em metros, × m/px)
- A penalidade depende do vértice de chegada: w(u, v) ≠ w(v, u); por isso, o grafo é direcionado
- O custo $\ell(P) = \Sigma\, w(e_i)$ não é uma distância: soma comprimentos e penalidades
- Fluxo: Graph (com pesos) → algoritmo → PathResult → post_processing → RobotPath → ros_bridge → Gazebo



Forma da penalidade p(v) em função de d(v) (ilustração da definição)

O parâmetro L define a preferência do planejador pelo centro dos corredores; a distâncias superiores a L pixels, o obstáculo não influencia o peso.

Algoritmos de busca em grafos

Os quatro algoritmos clássicos implementados e o papel de cada um neste projeto

BFS

O(|V| + |E|)

Busca em largura: minimiza o número de arestas. Tende ao caminho geometricamente mais curto, mas ignora a penalização e passa rente aos obstáculos.

DFS

não garante o menor caminho

Busca em profundidade: percorre cada ramo até o fim antes de retroceder. Utilizada apenas como referência de comparação; em grade, percorre quase toda a área transitável.

Dijkstra

O((|V| + |E|) log |V|)

Encontra o caminho de menor custo (pesos não negativos), examinando os vértices em ordem crescente de custo acumulado, em todas as direções.

A*

ótimo se h não superestimar

Semelhante ao Dijkstra, mas guiado por uma heurística h(n): a distância euclidiana até o destino. Por isso, explora menos vértices.

$$f(n) = g(n) + h(n)$$

g(n): custo desde a partida; h(n): estimativa até o destino

Caminho P e custo ℓ(P): definições gerais (todos os algoritmos)

$$P = v_0,\ e_1,\ v_1,\ \ldots,\ e_k,\ v_k \qquad \ell(P) = \sum_{i=1}^{k} w(e_i)$$

k + 1: número de vértices do caminho P; $w(e_i)$: peso da aresta e_i

Menos vértices explorados ≠ menor tempo: para cada vizinho, o A* calcula h(n), que envolve uma raiz quadrada; em grafos grandes e regulares, como os obtidos de imagens, esse custo adicional pode anular o ganho.

Árvore de Exploração Rápida Aleatória (RRT)

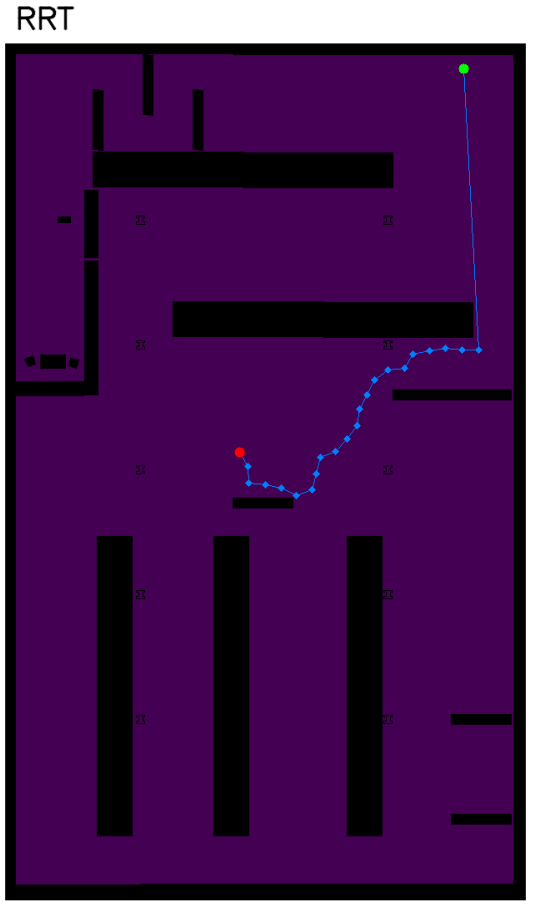
Planejador baseado em amostragem aleatória (LaValle, 1998): uma árvore cresce a partir da partida, explorando o espaço livre

1. Amostra aleatoriamente um ponto do mapa de navegabilidade
2. Seleciona o vértice da árvore mais próximo desse ponto
3. Avança, a partir desse vértice, um passo de tamanho fixo em direção ao ponto amostrado
4. Se o segmento não atravessa obstáculos, adiciona o novo vértice à árvore

Critério de parada: novo vértice com linha de visada desobstruída até o destino. **Viés para o objetivo:** em uma fração das amostras, o ponto amostrado é o próprio destino.

Vantagens: iterações de baixo custo computacional; probabilisticamente completo; reprodutível com semente fixa

Limitações: caminho não ótimo e irregular; dificuldade em passagens estreitas



Exemplo no mapa do galpão simulado: vértices da árvore em círculos

Comparabilidade do custo

O RRT não utiliza as arestas nem os pesos do grafo: seu custo é apenas o comprimento geométrico do caminho, sem as penalidades usadas pelos demais algoritmos.

Obstáculos móveis

Planejamento rápido, sem reconstrução do grafo a cada mudança: candidato ao replanejamento em ambientes dinâmicos (trabalho futuro).

Algoritmos sobre a imagem aérea do Outeiro da Glória

Imagem segmentada de 1000 × 750 px: ~440 mil vértices e 3,3 milhões de arestas; mesma partida e chegada para todos os algoritmos

Algoritmo	Vértices (k+1)	Custo ℓ(P)	Vért. expl. / iter.	Tempo (s)	Distância (pixels)
BFS	351	12.838,8	179.546	0,131	398,0
DFS	65.282	1.314.372,7	248.479	0,159	92.241,7
Dijkstra	879	1.462,2	228.525	0,392	990,7
A*	879	1.462,2	203.472	0,426	990,7
RRT	30	—*	289	0,010	931,8

Tabela 2. k+1: número de vértices do caminho P obtido; ℓ(P): custo do caminho P, soma dos pesos das arestas (mínimo para Dijkstra e A*); Distância: comprimento geométrico de P, em pixels; Vért. expl./iter.: vértices explorados (iterações, no RRT). * O RRT não utiliza a função de peso: custo ponderado não aplicável.

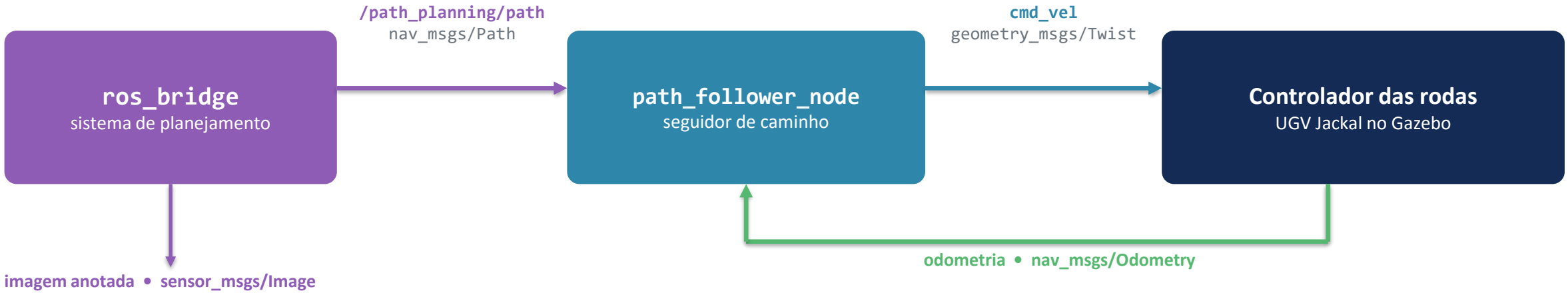
- **A* e Dijkstra:** mesmo custo mínimo (1.462,2); o A* explora cerca de 11% menos vértices, mas não foi mais rápido
- **BFS:** menor distância (398,0 px), porém custo ponderado muito maior: passa rente aos obstáculos
- **DFS:** 65.282 vértices e distância cerca de 100× a do A*
- **RRT:** menor tempo de planejamento (0,010 s, 289 iterações)



Figura 2: amarelo: não transitável; partida em vermelho, chegada em verde. DFS omitida (encobriria os demais caminhos).

ROS 2 e simulação no Gazebo

Processos independentes (nós) trocam mensagens por canais nomeados (tópicos)



- **Nó:** processo com uma função específica (ler um sensor, planejar, comandar os motores); não se confunde com vértice de grafo
- **Publicação e assinatura:** o emissor não precisa conhecer o receptor; assim, o UGV simulado pode ser substituído por um real sem alterar o planejamento
- **Gazebo:** simulador de código aberto; mundos e robôs descritos em arquivos SDF (posições e geometrias dos objetos)

Política de entrega de mensagens (QoS)

volatile: a mensagem é entregue apenas aos assinantes já conectados.

transient local: a última mensagem é armazenada e reenviada a assinantes tardios.

O planejador publica o caminho uma única vez e encerra; com transient local, o seguidor de caminho o recebe assim que os dois nós se conectam.

Lei de controle do seguidor de caminho

Nó path_follower_node (bloco Simulação Gazebo): a cada ciclo, compara a posição estimada do UGV com o próximo ponto de passagem (x_t, y_t)

Erro de orientação: ângulo que o UGV precisa girar, no intervalo [−π, π]

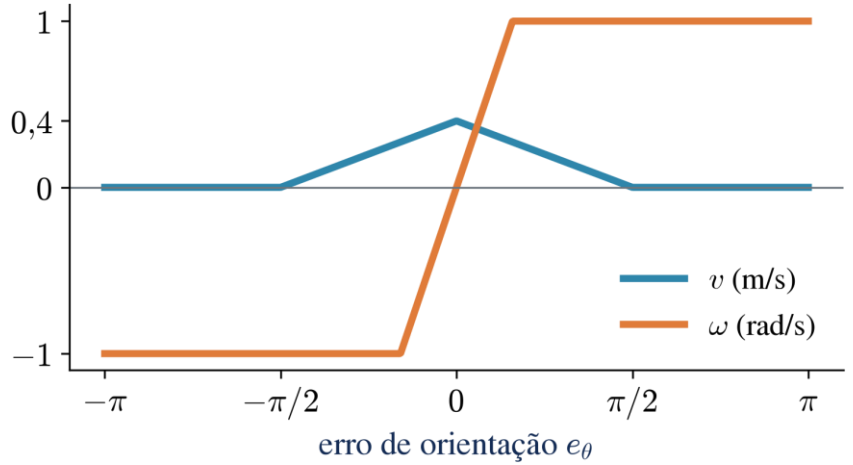
$$e_\theta = \operatorname{atan2}(\sin(\theta_d - \theta),\ \cos(\theta_d - \theta)), \quad \theta_d = \operatorname{atan2}(y_t - y,\ x_t - x)$$

O uso de seno e cosseno garante que o giro ocorra sempre pelo menor ângulo.

Comandos de velocidade (geometry_msgs/Twist)

$$\omega = \operatorname{sat}(2\, e_\theta,\ \omega_{\max}), \qquad v = v_{\text{cruzeiro}} \cdot \max\left(0,\ 1 - \frac{|e_\theta|}{\pi/2}\right)$$

A velocidade angular ω cresce com o erro, até o valor máximo; a velocidade linear v diminui com o desalinhamento e se anula acima de 90°: o UGV não avança enquanto não estiver alinhado.



v e ω em função do erro de orientação, com os parâmetros adotados

Parâmetros configurados no seguidor de caminho

0,4 m/s	1,0 rad/s	0,12 m	0,15 m
velocidade de cruzeiro	velocidade angular máxima	tolerância em cada ponto	tolerância na chegada

Um ponto de passagem é considerado alcançado quando a distância entre o UGV e o ponto é inferior à tolerância correspondente.

Mapa de navegabilidade gerado a partir do mundo simulado

Script gazebo_world_to_occupancy.py: converte o mundo do Gazebo em um mapa de navegabilidade no formato do sistema

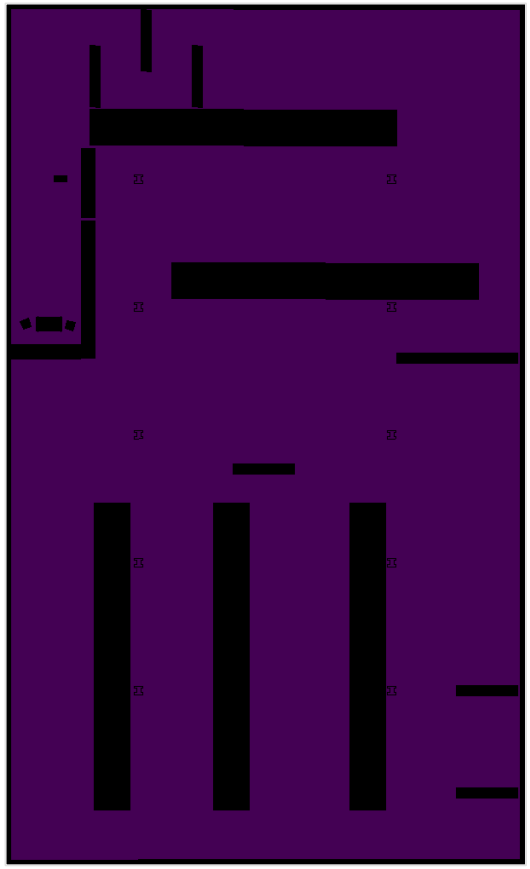
Problema: a imagem e o mundo simulado eram cenários distintos; a escala e a origem eram ajustadas manualmente, e o UGV executava a trajetória com a forma correta, porém deslocada ou fora de escala.

1. **Leitura do mundo**
   lê todos os objetos do arquivo SDF, inclusive os incluídos de outros diretórios e do repositório de modelos

2. **Posição de cada objeto**
   compõe as poses das partes de cada objeto para obter sua posição no referencial do mundo

3. **Projeção vertical**
   caixas, cilindros e malhas 3D projetados no plano do chão; pisos e tetos descartados; objetos não encontrados ignorados, com aviso

4. **Geração do mapa**
   área livre na cor transitável e obstáculos em preto; registro da resolução (m/px) e da origem (m) exatas



~30 × 50 m
galpão fechado

0,05 m/px
resolução

863 × 1242
pixels

Figura 3: mapa de navegabilidade do galpão simulado (mundo warehouse). Roxo: área livre; preto: obstáculos.

Resultado: erros do planejador deixam de se confundir com erros de alinhamento imagem–mundo.

Algoritmos sobre o mapa do galpão simulado

Mundo warehouse do Gazebo, 863 × 1242 px, 0,05 m/px: 502.833 vértices e 3.753.214 arestas; tempo: mediana de 3 execuções

Algoritmo	Vértices (k+1)	Custo ℓ(P)	Vért. expl. / iter.	Tempo (s)	Distância (pixels)
A* (ponderado)	609	28.714,2	391.239	2,173	709,1
Dijkstra (ponderado)	609	28.714,2	397.659	1,694	709,1
BFS	599	44.174,4	414.703	0,674	722,3
DFS	2.614	233.982,4	467.072	0,843	3.693,7
RRT	23	—*	230	0,073	757,3

Tabela 3. Mesmas definições da Tabela 2. * O RRT não utiliza a função de peso (custo ponderado não aplicável); com semente fixa, as 3 execuções produziram o mesmo caminho.

- **A* e Dijkstra:** caminhos ligeiramente diferentes, de mesmo custo (28.714,2), pelo centro dos corredores
- **BFS:** custo 54% maior, com trajeto rente às bordas
- **DFS:** distância cerca de 5× a do A*
- **RRT:** 23 vértices e 757,3 px, contra 709,1 px do A*, com mudanças bruscas de direção

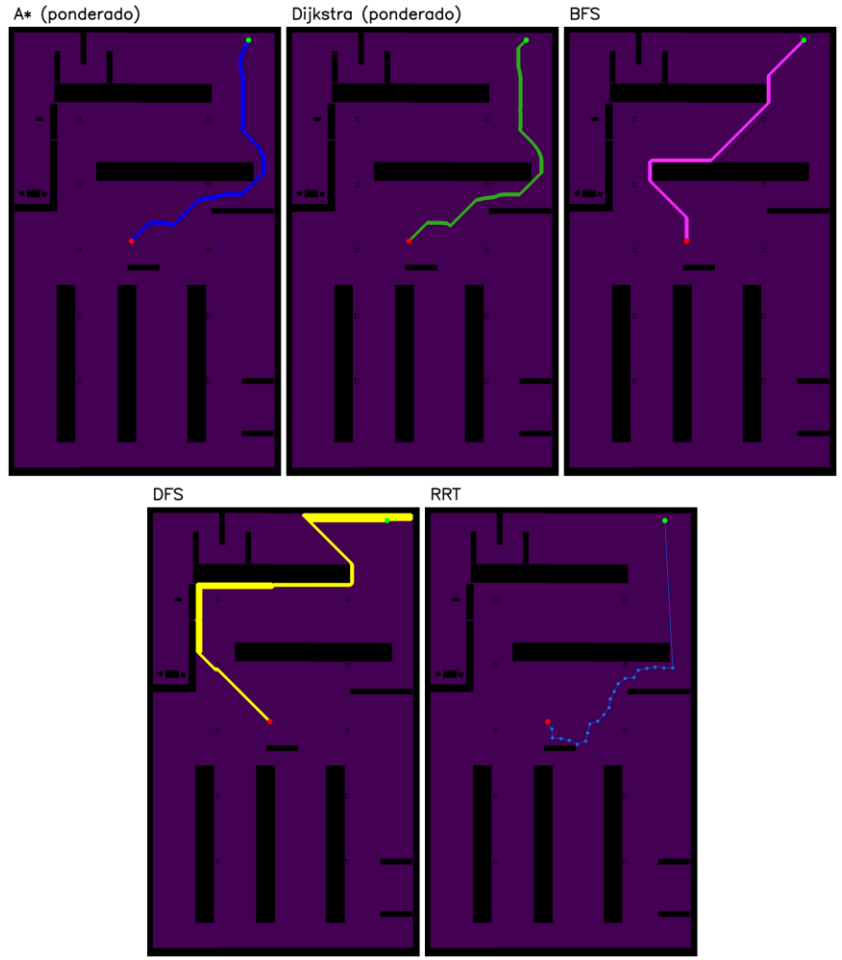
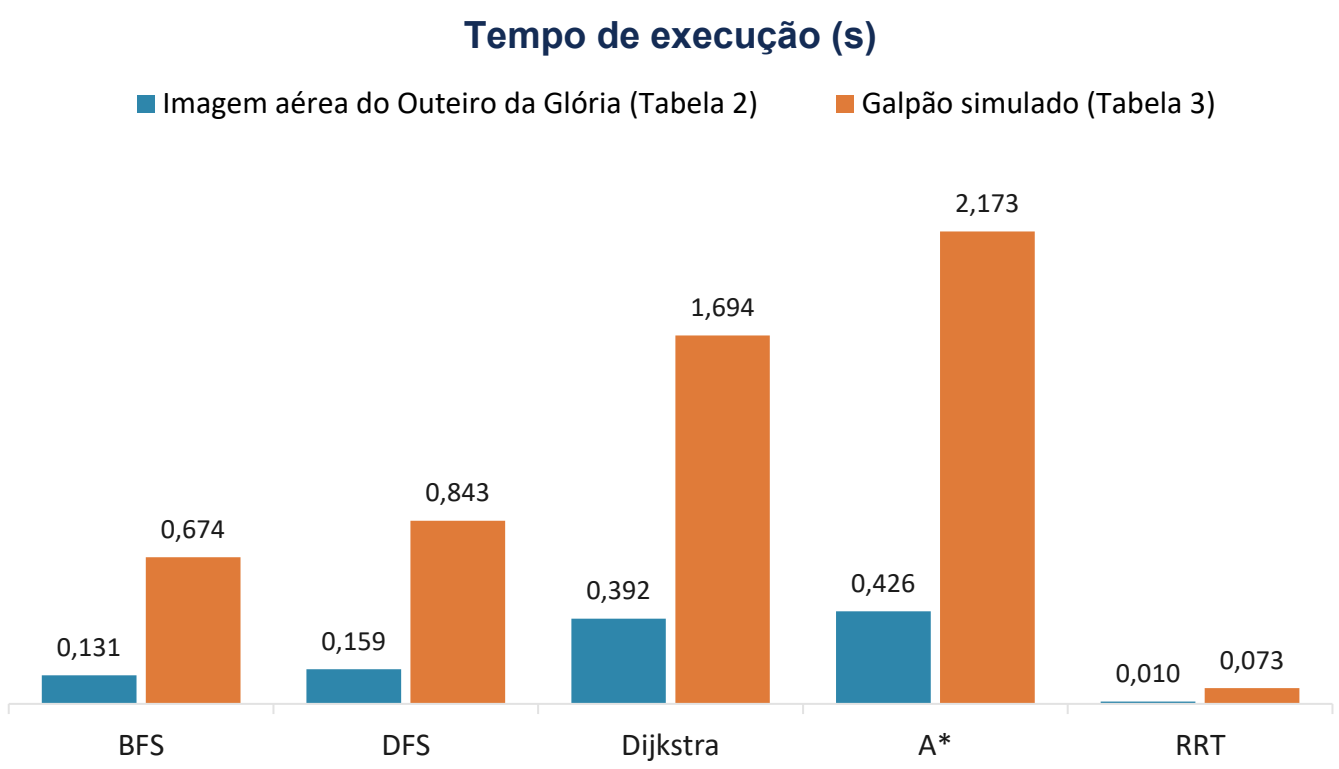


Figura 4: caminhos desenhados separadamente (recorte da área interna do galpão).

Algoritmos de busca em grafos × RRT: tempo e forma do caminho

Tempos de planejamento na imagem aérea do Outeiro da Glória e no galpão simulado; decisão para os experimentos



9 a 43× menor tempo de planejamento do RRT em relação aos algoritmos de busca em grafos, nos dois cenários

Desvantagens da solução dada pelo RRT, comparada à busca em grafos (Dijkstra e A*)

- Dijkstra e A*: curvas suaves, que acompanham a geometria dos corredores
- RRT: segmentos curtos com mudanças bruscas de direção, sem relação com os obstáculos, e um segmento final longo
- Os desvios dependem do cenário: traçado quase reto na área aberta; desvios pronunciados no galpão
- Mais apropriado a cenários abertos, com poucos obstáculos e sem passagens estreitas
- Cada mudança brusca exige que o UGV reduza a velocidade e gire no próprio eixo

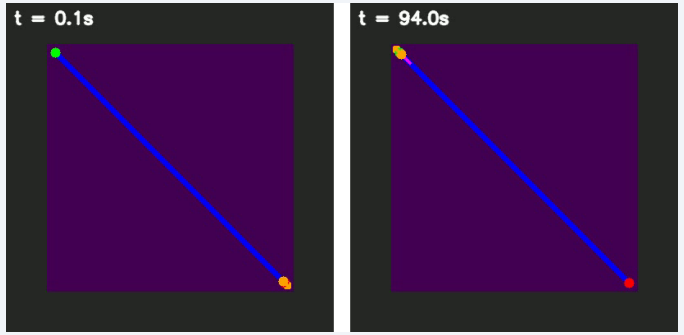
Decisão: apesar de o RRT planejar mais rapidamente, os experimentos no simulador consideram o algoritmo A* com pesos.

Validação no simulador Gazebo

Experimentos de ponta a ponta: o UGV executa o caminho recebido e para ao alcançar o destino

Condições gerais dos experimentos: mundo do galpão simulado (warehouse) no Gazebo, sem interface gráfica; caminho do A* com pesos enviado pela ponte ROS 2; velocidade de cruzeiro de 0,4 m/s; rastro do UGV desenhado a partir da odometria.

✓ UGV alcançou o destino



1 Mapa sintético

Teste básico: caminho em linha reta diagonal, sem obstáculos. Verifica o envio do caminho, o seguimento e a parada no destino.

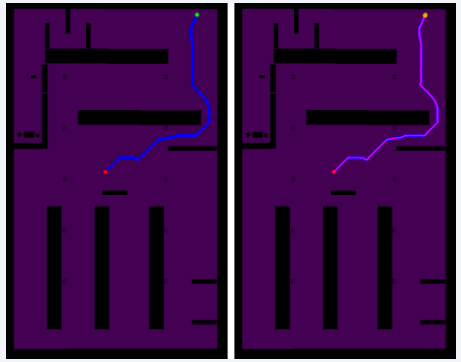
✓ UGV alcançou o destino



2 Imagem aérea

Caminho planejado sobre a imagem aérea do Outeiro da Glória, segmentada pela U-Net (MobileNet V2).

✓ UGV alcançou o destino



3 Galpão simulado

Mapa de navegabilidade gerado a partir do próprio mundo simulado: único experimento em que o caminho desvia dos obstáculos do mundo.

Experimento 2: imagem aérea do Outeiro da Glória

Imagem aérea do Outeiro da Glória redimensionada para 800 × 600 px e segmentada pela U-Net com MobileNet V2



Figura 5: (a) entrada: caminho do A* (azul), partida em vermelho e chegada em verde; (b) resultado (t = 233,1 s): UGV simulado (laranja) na chegada e rastro recente em magenta.

Da rua estreita (alto) ao pátio pavimentado (embaixo), contornando edificações e veículos. Duração do registro: 232,9 s, incluindo o deslocamento inicial até o primeiro ponto. O experimento avalia o seguimento do caminho, e não o desvio dos obstáculos do mundo simulado.

Experimento 3: galpão simulado, com o mapa do próprio mundo

Mesmos pontos de partida e chegada da Tabela 3, com folga mínima de 12 px (0,6 m), compatível com as dimensões do Jackal

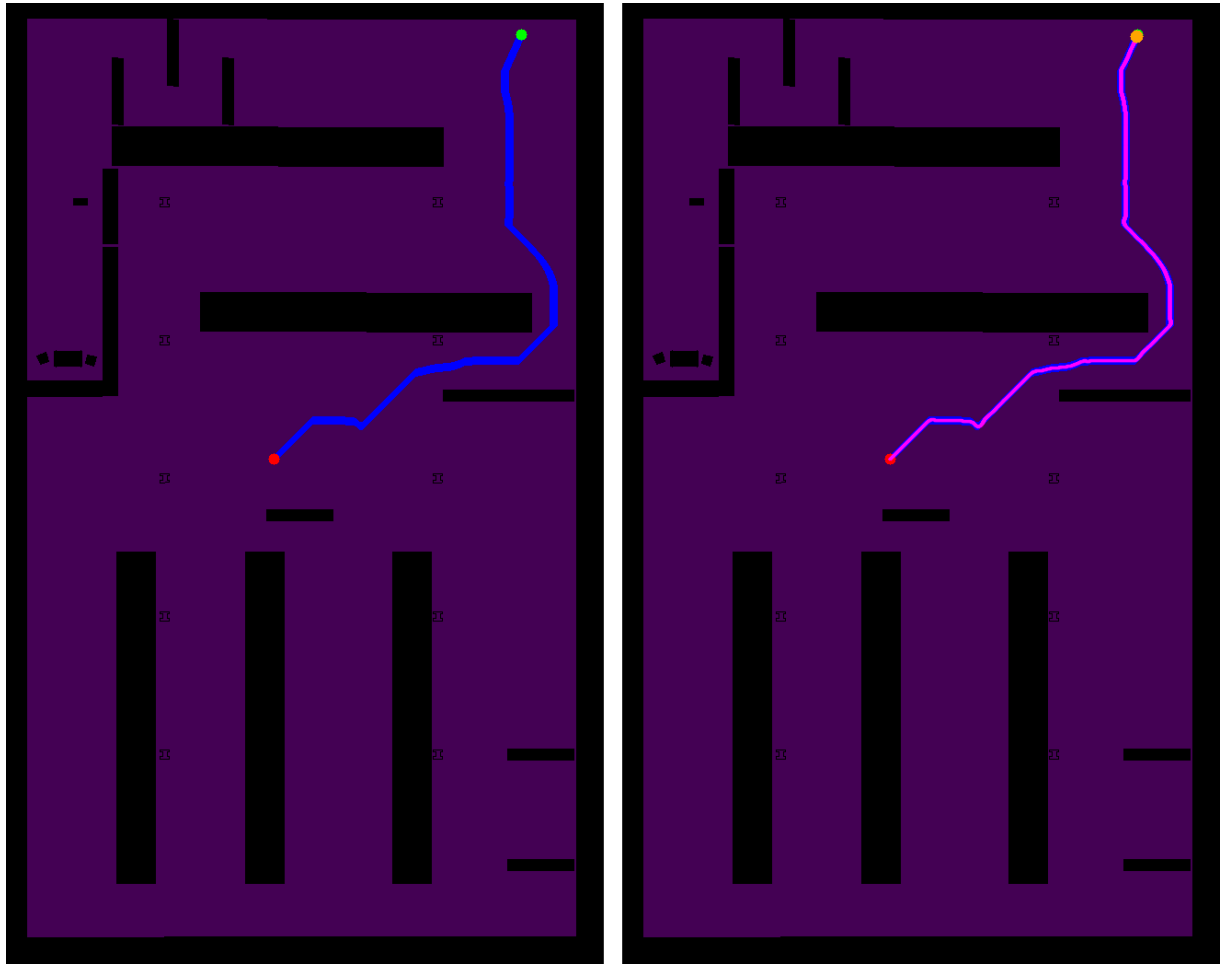


Figura 6: (a) caminho planejado pelo A*; (b) rastro completo da odometria (magenta) e posição final (laranja).

621 pontos de passagem	**35,8 m** caminho planejado
35,1 m percorridos	**95,8 s** de movimento
0,37 m/s velocidade média	**4.537** amostras de odometria (~47/s)

Único experimento em que o mapa corresponde exatamente ao mundo: o caminho desvia dos obstáculos do mundo simulado. O UGV partiu do primeiro ponto do caminho; velocidade média próxima à de cruzeiro (0,4 m/s).

Experimento 3: erro lateral em relação ao caminho planejado

Caminho do A* com pesos, no galpão simulado. Erro lateral: menor distância de cada posição registrada ao caminho planejado

0,8 cm
erro lateral médio

1,3 cm
raiz do erro quadrático médio

2,9 cm
percentil 95 do erro lateral

7,1 cm
erro lateral máximo

Os quatro indicadores são inferiores à tolerância dos pontos (0,12 m). O UGV parou a 0,15 m do ponto de chegada, valor igual à tolerância de chegada configurada.

Localização dos maiores desvios: nas mudanças de direção, onde o caminho planejado forma um vértice com ângulo acentuado e o UGV descreve uma curva.

Limitação da métrica: a posição do UGV é obtida da odometria, a mesma estimativa usada pelo controlador. Assim, o erro lateral avalia a fidelidade do seguimento do caminho, e não a precisão da localização; em um robô real, os erros de odometria se acumulariam. Nos Experimentos 1 e 2, a odometria não foi registrada: a avaliação foi apenas visual.

Conclusões: o sistema

Avanços em relação ao PFC 2025: segmentação integrada e execução do caminho por um UGV simulado

Arquitetura modular
Cinco módulos independentes: novos algoritmos ou modelos são incorporados sem alterar os demais.

RRT incorporado
RRT e algoritmos de busca em grafos comparados sobre os mesmos mapas.

Segmentação integrada
U-Net pré-treinada: o sistema parte de imagens aéreas reais capturadas por UAV, e não de mapas já segmentados.

ROS 2 + Gazebo
O caminho planejado passou a ser executado por um UGV simulado.

Correspondência de escala e origem
Mapa de navegabilidade gerado a partir do próprio mundo simulado, sem ajuste manual de escala e origem.

3 de 3

experimentos em simulação concluídos, com o UGV alcançando o destino em todos

< 1 cm

de erro lateral médio entre o caminho executado e o planejado (Experimento 3, galpão simulado)

Passos futuros: validação em ambiente real

Prevista para os dois últimos meses do cronograma, a etapa não foi realizada; é o primeiro passo para a continuidade do trabalho

1 Plataforma

Exigiria um UGV com odometria confiável, unidade inercial e interface ROS 2 disponível para uso continuado no período de ensaios, o que não pôde ser assegurado no prazo.

2 Espaço e imagem

Espaço aberto com obstáculos permanentes e percursos de dezenas de metros, e voo de UAV com autorização, meteorologia adequada e medição de referência da escala; eventual repetição do voo.

3 Tempo

A integração com o ROS 2 e com o simulador consumiu parte significativa do prazo: comunicação entre nós, configuração do simulador e, principalmente, falhas da interface gráfica.

Verificado em simulação (malha fechada)

Envio do caminho, seguimento dos pontos de passagem, correção pela odometria e parada no destino, com as mesmas interfaces de um UGV real.

Aspectos que exigem ensaio real

Terreno irregular e escorregadio, erros de odometria acumulados em percursos longos, erros de georreferenciamento da imagem e obstáculos ausentes na imagem aérea.



Clearpath Jackal real • Foto: U.S. Army (domínio público)

Perspectivas futuras

Em ordem aproximada de prioridade

1

Validação em ambiente real

UGV físico, imagem aérea da área de teste e escala medida em campo, para avaliar terreno irregular e erros de odometria

2

Erro lateral médio por algoritmo

Medido neste trabalho apenas para o A*; estender a medição a Dijkstra, BFS, DFS e RRT, comparando a facilidade de execução dos caminhos

3

Ambientes dinâmicos

Replanejamento em tempo real com RRT e variantes, ou algoritmos de caminhos mínimos em grafos dinâmicos (algoritmos incrementais); 1ª perspectiva do PFC 2025

4

Múltiplas execuções do RRT

Comparar os algoritmos de busca em grafos com o valor médio de múltiplas execuções do RRT (comprimento, tempo e mudanças de direção)

5

Suavização do caminho

Substituir as mudanças bruscas de direção por curvas executáveis pelo UGV; benefício maior para o RRT e a BFS

6

Redução do armazenamento do grafo

Cálculo implícito das arestas e de seus respectivos pesos, sem armazenar os valores correspondentes

Resumo dos objetivos alcançados neste projeto

Arquitetura modular
5 módulos; algoritmos e modelos substituíveis

Segmentação semântica
U-Net identifica as áreas transitáveis

Algoritmos de planejamento
BFS, DFS, Dijkstra, A* e RRT comparados; A* com pesos adotado para o UGV

ROS 2 + Gazebo
3 experimentos concluídos; erro lateral médio de 0,8 cm (Experimento 3)

Trabalhos futuros: validação em ambiente real, com UGV físico, e demais perspectivas apresentadas.

Obrigado.

Gabriel Dias Vilela • Tassandro Cavalcante Leitão

Uso de IA generativa: o assistente Claude (Anthropic), por meio do Claude Code, foi utilizado como apoio na elaboração, na revisão textual e na formatação destes slides; não foi utilizado para gerar dados ou resultados.